

CLASS NOTES · COMPLETE SERIES

NETWORKS, FAST

ADDRESSING · TRANSMISSION MEDIA · THE DATA LINK LAYER · MEDIUM ACCESS
CONTROL

Eleven chapters of class notes, each one a screen or two — addressing, the media, framing, the sliding windows, and the whole of medium access. **Every chapter ends in a five-line summary that is the entire chapter**, and the switch in the margin hides everything except those lines. Read it through once, then read only the summaries.

11 CHAPTERS	48 SECTIONS	14 DIAGRAMS	58 SUMMARY LINES
-----------------------	-----------------------	-----------------------	----------------------------

CHAPTER 01

ADDRESSING IN A NETWORK

Before a single bit can be delivered, a network has to be able to name three different things: the machine, the network interface, and the program running inside the machine. This chapter covers the three address types, and then how a switch decides — purely from the destination MAC address — whether a frame is for one station, a group, or everybody.

1.1 The three types of address

Delivery is a layered job, and each layer needs its own kind of address.

- 1 IP address** — the logical address of a host. Used for host-to-host delivery across networks, so it is the address a router works with.
- 2 MAC address** — also called the physical or hardware address. It is 48 bits and is burned into the NIC. Used for node-to-node delivery inside one LAN.
- 3 Port address** — identifies the particular process inside the host, so the arriving data reaches the right program.

KEY LIMITATION OF THE MAC ADDRESS

A MAC address is used only inside a local area network. A frame addressed by MAC never leaves the LAN. When the destination is on another network it is the **IP address** that makes delivery possible, and a fresh MAC address is used on every hop.

ADDRESS	LENGTH / FORM	SCOPE OF DELIVERY	USED BY
IP address	Logical, assigned by software	Host to host, across networks	Router — network layer
MAC address	48 bits, fixed in the NIC, six hex octets	Node to node, inside one LAN only	Switch — data link layer
Port address	A number identifying a process	Process to process	Transport layer

1.2 Structure of a MAC address

A MAC address is 48 bits, written as six hex octets, for example `00:1A:2B:3C:4D:5E`. Two bits of the **first** octet carry meaning of their own, and they are the two bits a switch inspects.

```

first octet  b7 b6 b5 b4 b3 b2 b1 b0
                ^ bit 0 : 0 = unicast, 1 = multicas
                ^ bit 1 : 0 = universal, 1 = local
  
```

- **bit 0 = 0** → unicast · **bit 0 = 1** → multicast
- **bit 1 = 0** → universally administered, the address the manufacturer gave it · **bit 1 = 1** → locally administered, set by software

1.3 How a switch classifies a frame

The test is made only on the destination MAC address, and it is made in a fixed order.

THE RULE

1. If the destination is `FF:FF:FF:FF:FF:FF` — all 48 bits are 1 — it is **broadcast**.
2. Otherwise look at bit 0 of the first octet: 0 → unicast, 1 → multicast.

WORKED EXAMPLE

Convert the first octet to binary and read its last bit.

00 : 1A : ...	00 = 0000 0000	bit 0 = 0	unicast
01 : 00 : 5E : ...	01 = 0000 0001	bit 0 = 1	multicast
03 : ...	03 = 0000 0011	bit 0 = 1	multicast
FF : FF : FF : FF : FF	FF = 1111 1111	all ones	broadcast

Both **01** and **03** end in binary 1, so both are multicast even though they look nothing alike in hex. The all-ones address is tested first, because it would otherwise also pass the multicast test.

EXAM TIP

Only the **last** bit of the **first** octet decides unicast versus multicast. Everything else in the address is irrelevant to that question.

CHAPTER 01 AT A GLANCE

- Three addresses: **IP** host to host across networks, **MAC** node to node inside one LAN, **port** process to process.
- A MAC address cannot move data to a different network — that is the IP address's job, and a new MAC is used on every hop.
- A MAC address is **48 bits**, six hex octets, burned into the NIC.
- Classification: all ones → broadcast; otherwise bit 0 of the first octet, 0 → unicast, 1 → multicast.
- Bit 1 of the first octet: 0 → universal, 1 → locally administered.

CHAPTER 02

SWITCHES, BRIDGES AND THE LOOP PROBLEM

A switch does not flood every frame everywhere. It keeps a small table that tells it which interface leads to which MAC address. This chapter explains how that table is built automatically, why the entries have to be refreshed, what goes wrong when two bridges are joined by more than one path, and how the BPDU mechanism repairs it.

2.1 The switch forwarding table

A switch keeps a table that maps a MAC address to the interface (port) on which that address can be reached. The switch sends every arriving packet according to this table.

MAC ADDRESS	INTERFACE	STATE
AA	P1	Fresh
DD	P2	Fresh

TERMINOLOGY

A **bridge** and a **switch** are the same thing for the purposes of this discussion — a switch is simply a bridge with many ports. The words are used interchangeably below.

2.2 Learning bridges

The table is not configured by hand. A bridge fills it in by itself, which is why it is called a **learning bridge**. The idea is simple: the source address of every arriving frame tells the bridge where that station is.

2.2.1 How a frame is handled, step by step

- 1 Learn from the source address.** The frame arrives at B1 on port P1 carrying SA = DD . B1 records the entry DD → P1 in its table. It now knows how to reach A in future.
- 2 Look up the destination address.** B1 searches its table for DA = AA .
- 3 Forward or flood.** If AA is present in the table, the frame is sent only on the recorded interface. If it is not present, the bridge **floods** the frame on every port except the one it arrived on, so that the frame still reaches its destination.
- 4 The next bridge repeats the process.** B2 receives the same frame, learns DD → P3 (the port facing B1), and delivers the frame towards D.
- 5 The reply completes the learning.** When D answers, its source address AA is learned by B2 and then by B1, and from that point onwards traffic in both directions is forwarded, not

flooded.

WHY ENTRIES HAVE TO BE REFRESHED

Each entry is marked **Fresh** when it is created. A station may be unplugged and moved to a different port, so an old entry would become wrong. Every table entry therefore keeps refreshing after some time: it is renewed each time a frame is seen from that address, and it is deleted if no such frame arrives before the timer expires. The bridge then re-learns the address from scratch.

2.3 The loop problem

Bridges are usually connected together with more than one link so that the network survives a cable failure. But this creates a physical **loop**, and a loop is fatal to the flooding rule described above.

When two ports connect the same two bridges, a broadcast or an unknown-destination frame is flooded by B1 to B2 on port 1; B2 floods it back to B1 on port 2; B1 floods it again on port 1, and so on. The frame is copied around the ring endlessly.

CONSEQUENCE: INFINITE FORWARDING

The same frame is forwarded for ever, multiplying at each pass. The bandwidth of the LAN is consumed by copies of a single frame, and the bridge tables are corrupted because the same source address keeps appearing on different ports.

2.4 The solution: BPDU and port blocking

The loop is removed by the software itself, not by re-cabling. The bridges exchange a special control frame among themselves and agree, by priority, which of the redundant ports is allowed to work.

BPDU – BRIDGE PROTOCOL DATA UNIT

A control frame exchanged between bridges.

- **Check whether a loop exists or not.** A bridge that receives its own BPDU back knows there is a loop.
- **Carry configuration and priority information**, so that the bridges can agree on which one of them takes precedence.

As a result, only one port is allowed to work; the other port gets **blocked**.

EXAM TIP

Note the chain of reasoning: two paths → loop → infinite forwarding → BPDU detects the loop → priority decides → one port is blocked. The redundant link is not removed, it is only disabled, so it can be brought back if the working link fails.

CHAPTER 02 AT A GLANCE

- A switch holds a table of **MAC address** → **interface** and forwards packets according to it. Bridge and switch mean the same thing here.
- A **learning bridge** fills that table automatically from the source address of arriving frames; unknown destinations are flooded.
- Table entries are marked fresh and keep refreshing after some time, so that stale entries disappear.
- Two ports joining the same bridges form a **loop**, which causes infinite forwarding.
- **BPDU** (Bridge Protocol Data Unit) checks whether a loop exists and carries configuration and priority; only one port stays active and the other is blocked.

CHAPTER 03

DATA TRANSMISSION AND NETWORK PARAMETERS

Once the path exists, the next question is how much traffic it can actually carry and how long each packet takes. This chapter introduces the bottleneck link, the relation between offered load and link capacity, the components of delay at a router, and the reason a router must maintain queues at all.

3.1 The bottleneck link

A path from a sender to a receiver passes through several routers. Each link on that path has its own capacity, and **the slowest link decides the performance of the whole path**. That link is called the *bottleneck link*.

OFFERED LOAD

The offered load is the amount of traffic the sources **present** to the link, as opposed to the amount the link can actually carry.

If the bottleneck link is 100 Mbps, then in theory an offered load above 100 Mbps means that packets get dropped. In practice the boundary is not quite so sharp.

WHY THE LOAD MAY EXCEED THE CAPACITY SLIGHTLY

If the link is 100 Mbps, the offered load can be just a little more than 100 Mbps without loss, because while new packets are arriving some packets are simultaneously being transmitted and are leaving the buffer. The buffer absorbs the short-term excess. For dropping not to occur, the **long-run** offered load must stay at or below the capacity of the link.

offered load < capacity → queue stays short → no loss
offered load > capacity → queue grows → buffer fills → pa

3.2 Components of delay

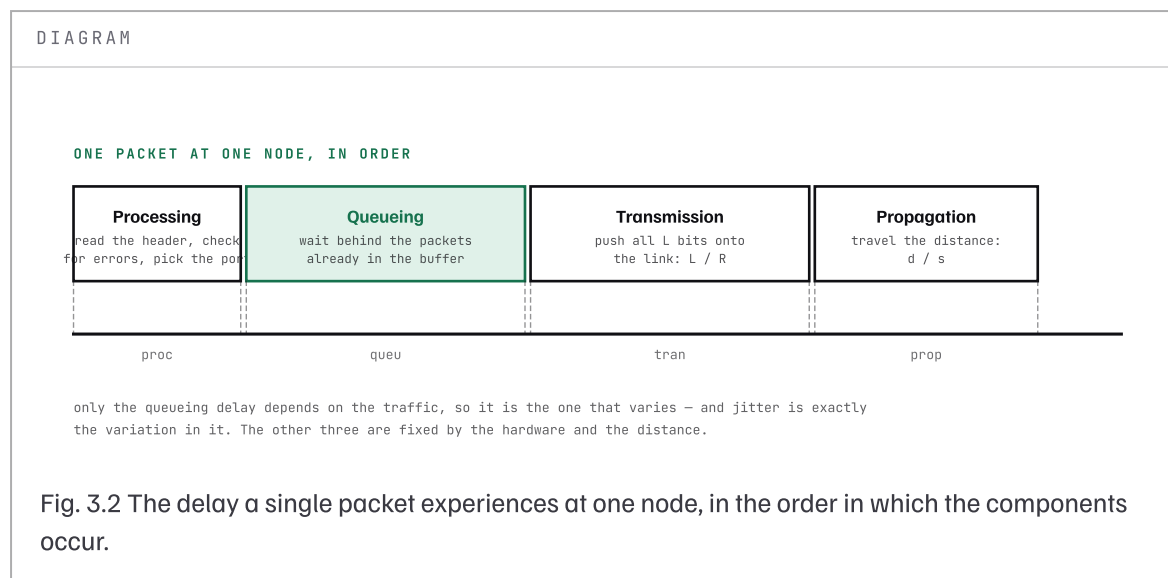
The total time a packet spends at one node is the sum of several separate delays. The first two are the ones that arise inside the router itself; the last two arise on the link leaving it.

- 1 Processing delay** — the time the router takes to examine the packet header, check it for errors and decide the outgoing interface.
- 2 Queueing delay** — the time the packet waits in the buffer while packets ahead of it are transmitted. This is the delay that grows as the offered load approaches the link capacity.
- 3 Transmission delay** — the time taken to push all the bits of the packet onto the link. It depends on the packet length and the link rate.

- 4 Propagation delay** — the time the signal takes to travel along the physical medium from one end of the link to the other. It depends on the distance and the medium, not on the packet size.

$$d_{\text{total}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

Only the **queueing** term depends on how busy the network is. The other three are essentially fixed for a given packet, router and link.



3.3 Why queueing is necessary

QUESTION ASKED IN CLASS

Why is queueing necessary? Because two packets can arrive at a router simultaneously. Without a queue the router cannot make out which packet is intended for whom — the two arrivals interfere with each other and the data becomes invalid. A link can carry only one packet at a time, so the second packet has nowhere to go and would be destroyed.

With a queue, packets P1 and P2 that arrive at the same instant are simply stored in the buffer and taken out one after another. Both are processed and sent correctly, one behind the other, on the outgoing link.

EXAM TIP

Queueing is the mechanism that makes simultaneous arrivals safe; queueing delay is the price paid for it. A router therefore always has buffers, and packets are dropped only when those buffers are completely full — which happens when the offered load stays above the capacity of the bottleneck link.

CHAPTER 03 AT A GLANCE

- The **bottleneck link** is the slowest link on the path; it fixes the usable rate of the whole path.
- Offered load above the link capacity leads to packet drops; a small short-term excess is absorbed by the buffer.
- Delay at a node = **processing + queueing + transmission + propagation**, and only queueing grows with load.
- Queueing is necessary because two packets can arrive at the same time; without a queue the router cannot tell which is which and the data becomes invalid.

CHAPTER 04

TRANSMISSION MEDIA

Data ultimately travels as a physical signal. This chapter covers the three primary forms that signal can take, and then the three media built around them: twisted-pair copper, coaxial cable and optical fibre — including why a twisted pair cancels noise, why a coaxial cable must be impedance matched, and why a fibre core must have a higher refractive index than its outer layer.

4.1 The three primary ways of transmission

- 1 **Electrical signal** — a varying voltage carried on a conductor.
- 2 **Electromagnetic waves (EMW)** — radiated energy, used for wireless transmission.
- 3 **Optical signal** — light guided through a transparent medium.

KEY POINT

Whatever form the signal takes on the medium, **inside the circuit it is always an electrical signal**. An optical or electromagnetic signal is converted into an electrical signal at the receiver before the equipment can process it, and converted back at the transmitter.

4.2 Twisted-pair (Ethernet) cable

The common LAN cable is a twisted-pair cable, also called an Ethernet cable, terminated with an RJ45 connector. Data is not sent as a single voltage on a single wire; it is sent as a **voltage difference between two wires**.

DIFFERENTIAL TRANSMISSION

The transmitter puts +ve on one wire and –ve on the other wire of the pair. The receiver does not read either wire on its own — it calculates the **difference** between them. The information is carried entirely by that difference.

IMPORTANT

Noise gets cancelled because the receiver works with a **difference**, not with an absolute voltage. The noise appears on both wires with the same sign and disappears in the subtraction; the wanted signal appears with opposite signs and survives. The essential idea is therefore: *transfer the voltage difference into the two wires*.

4.3 Coaxial cable

A coaxial cable carries the signal on a central conductor surrounded by an insulating layer and an outer conducting shield. It is the standard example of a **transmission line** — a cable long enough, relative to the signal wavelength, that its own electrical properties matter.

4.3.1 Impedance matching. A transmission line has a characteristic impedance, and the equipment at each end has its own impedance. These have to agree.

$Z_i = 50 \text{ ohm}$ $Z_o = 100 \text{ ohm}$ $\text{LOAD} = 50 \text{ ohm}$

Z_i = input impedance

Z_o = impedance of the line

LOAD = impedance at the far end

PROBLEM

If the input impedance is not matching the load, some problems occur: part of the energy is **reflected back** down the cable instead of being delivered, the reflected wave interferes with the signal that follows it, and the effective power that reaches the receiver falls.

Hence we always **match the load** — the source impedance, the cable impedance and the load impedance are chosen to be the same value (50Ω in the example above).

4.4 Optical fibre

An optical fibre guides light along its length. The middle part is called the **core**; around it is an outer layer, the **cladding**. The two layers are made of glass with different refractive indices, and that difference is what traps the light inside.

LAYER	REFRACTIVE INDEX	RELATIVE VALUE
Inner medium — the core	n_1	Bigger refractive index
Outer medium — the cladding	n_2	Smaller refractive index

THE CONDITION — TOTAL INTERNAL REFLECTION

When the outer refractive index is smaller than the inner one ($n_2 < n_1$), a ray striking the boundary at a shallow enough angle is completely reflected back into the core instead of escaping. This is **total internal reflection**, and it is what makes the light follow the fibre even round a bend.

NOTE – SENDING SEVERAL MESSAGES AT ONCE

We can send multiple messages by **varying the angle** at which the light is launched into the fibre. Each launch angle follows its own zig-zag path, and the receiver can also detect and separate them. This is only possible when the core is wide enough to allow more than one path.

4.4.2 The three types of optical fibre.

TYPE	CORE WIDTH	PATHS (MODES)	REFRACTIVE INDEX OF CORE	REMARK
1. Single mode	Very thin	One	Constant	Used generally
2. Multimode, step index	Bigger	Many	Constant, steps at the boundary	Different angles possible
3. Multimode, graded index	Bigger	Many	Keeps changing across the core	Costly

WHERE OPTICAL FIBRE IS USED

Optical fibre is the medium used for the very long, very high capacity links — for example, the submarine and terrestrial cables connecting countries to one another.

CHAPTER 04 AT A GLANCE

- Three primary ways of transmission: electrical signal, EMW, optical signal; inside the circuit it is always electrical.
- Twisted pair / RJ45: +ve on one wire, –ve on the other; the receiver reads the **difference**, so common noise cancels.
- Coaxial cable is a transmission line; Z_i , Z_o and the load must be matched ($50\ \Omega$), otherwise reflections and loss occur.
- Optical fibre: core (n_1) surrounded by cladding (n_2), with $n_2 < n_1$ giving total internal reflection.
- Three fibre types: single mode, multimode step index, multimode graded index. Varying the launch angle carries multiple messages — possible only in multimode.

CHAPTER 05

THE DATA LINK LAYER AND FRAMING

The physical layer delivers a stream of bits; it has no idea where one message ends and the next begins. The data link layer imposes that structure. This chapter covers the three services of the data link layer and then the first of them in detail — framing, by bit stuffing and by byte stuffing.

5.1 Services of the data link layer

The data link layer provides three distinct services. The first is the subject of this chapter; the other two are taken up in Chapters 6 and 7.

SERVICE	WHAT IT DOES	MECHANISM
Framing (data control)	Marks where each frame starts and ends in the bit stream	(i) Bit stuffing · (ii) Byte stuffing
Error control	Detects frames that arrived damaged, and gets them sent again	CRC, ARQ protocols
Flow control	Stops a fast sender from overrunning a slow receiver	Stop-and-Wait, sliding window

5.2 Framing

DEFINITION

Framing means marking off a block of data so the receiver can tell where it begins and where it ends. The sender places a special pattern — a **delimiter**, or **flag** — before and after the data.

delimiter → frame (payload) → delimiter

ASSUMPTION USED THROUGHOUT THIS CHAPTER

We assume there is **no noise** for these framing protocols. The only difficulty being solved here is the ambiguity of the delimiter, not corruption of bits. Noise is dealt with separately, by error control.

THE PROBLEM FRAMING MUST SOLVE

If the flag pattern also happens to occur **inside the payload**, the receiver will mistake it for the end of the frame and cut the frame short. The data must therefore be altered slightly before sending, so that the flag pattern can never appear by accident in the middle. That alteration is called **stuffing**, and it must be exactly reversible at the receiver.

5.3 Byte stuffing (character stuffing)

Byte stuffing works on whole characters. Recall that 1 byte = 8 bits = 1 character. Two characters are reserved.

- **FLAG CHARACTER — FG.** Written here as `~`. It marks the start and the end of the frame.
- **ESCAPE CHARACTER — ES.** Written here as `}`. It means “the next character is ordinary data, not a delimiter”.

- 1 **Stuff.** The sender scans the payload. Wherever it finds a flag character, it inserts an escape character in front of it.
- 2 **Frame.** The sender now adds a flag at the beginning and a flag at the end of the stuffed data.
- 3 **Transmit.** The receiver sees a flag, starts collecting, and stops at the next unescaped flag.
- 4 **De-stuff.** The receiver removes the opening and closing flags, and removes every escape character, keeping the character that follows it as plain data.

WORKED EXAMPLE — A FLAG INSIDE THE DATA

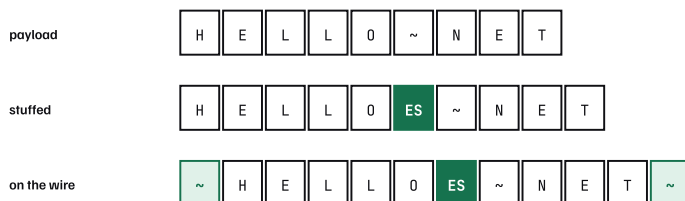
The sender wants to transmit the payload `HELLO~NET`. The `~` in the middle is the flag character, so it cannot be sent as it stands.

Payload	H	E	L	L	O	FG	N	E	T	the flag appears			
Stuffed	H	E	L	L	O	ES	FG	N	E	T	an ES is inserted		
Framed	FG	H	E	L	L	O	ES	FG	N	E	T	FG	this is what is
Recovered	H	E	L	L	O	FG	N	E	T	identical to the			

The delimiting flags and the inserted escape are removed at the receiver, so the recovered payload is exactly what was sent.

DIAGRAM

BYTE STUFFING – SENDING HELLO~NET WHEN ~ IS THE FLAG



the receiver drops the ES and keeps the byte after it, so the payload comes back byte for byte

Fig. 5.1 Byte stuffing of HELLO~NET. The delimiting flags and the inserted escape are removed at the receiver, so the recovered payload is exactly what was sent.

WORKED EXAMPLE – AN ESCAPE INSIDE THE DATA

The same treatment is applied to the escape character itself. If } occurs in the data — for example the payload HELLO}NET — then the escape is preceded by another escape.

Payload	H	E	L	L	O	ES	N	E	T	an escape appears			
Sent	FG	H	E	L	L	O	ES	ES	N	E	T	FG	ES ES means one

The receiver drops the first ES of the pair and keeps the second as ordinary data.

THE RULE IN ONE LINE

Before sending, insert an **ES** in front of every **FG** and every **ES** found in the payload. On reception, delete the first ES of any such pair. Because every inserted character is removed exactly once, the payload is recovered unchanged.

5.4 Bit stuffing

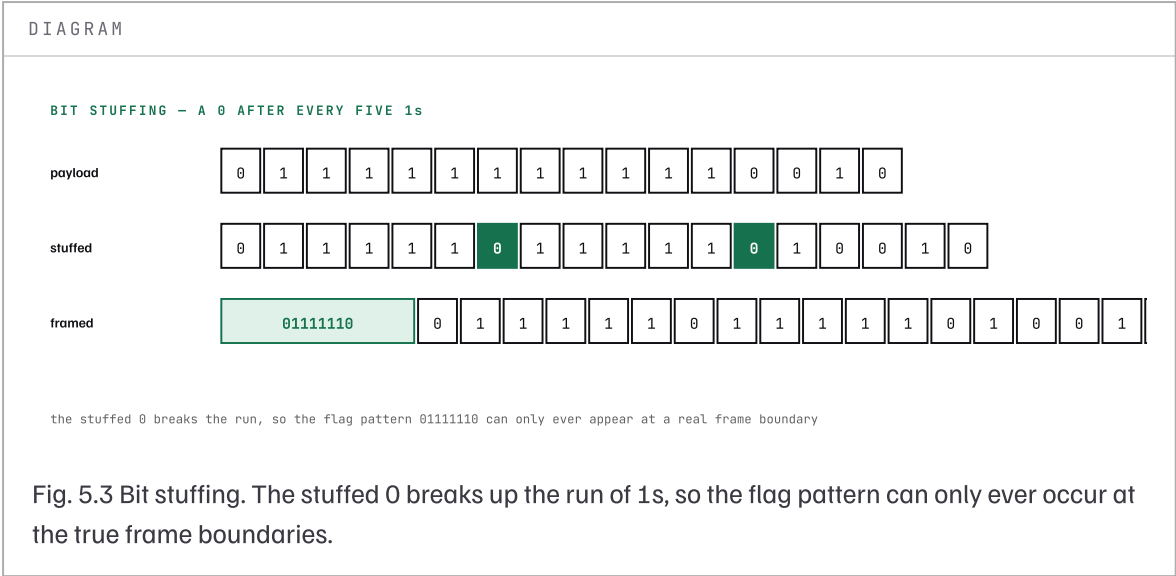
Byte stuffing works only when data is a whole number of characters. Bit stuffing does the same job at bit level, so that the payload may be an arbitrary sequence of bits.

THE METHOD

The flag is the bit pattern 01111110 — that is, six consecutive 1s between two 0s.

- **Sender:** while scanning the payload, after every **five** consecutive 1s it inserts a 0. The payload can therefore never contain six 1s in a row, so it can never imitate the flag.
- **Receiver:** after every five consecutive 1s it removes the following 0, restoring the original bits.

Payload	0 1 1 1 1 1 1 0 0 1	six 1s - looks like a flag
Stuffed	0 1 1 1 1 1 0 1 0 0 1	a 0 is inserted after five 1s
Framed	FG 0 1 1 1 1 1 0 1 0 0 1 FG	FG = 01111110



	BIT STUFFING	BYTE STUFFING
Works on	Individual bits	Whole characters (1 byte = 8 bits)
Delimiter	Flag pattern 01111110	Flag character (FG)
Inserted item	A single 0 bit after five 1s	An escape character (ES) before FG or ES
Payload length	Any number of bits	Must be a whole number of characters

EXAM TIP

Both techniques answer one question: how does the receiver know the flag it just saw is a real boundary? The answer is the same in both cases — the sender guarantees, by inserting something, that the flag pattern cannot occur inside the payload; the receiver removes exactly what was inserted.

CHAPTER 05 AT A GLANCE

- Data link layer services: **framing**, **error control** (CRC), **flow control**.
- A frame is delimiter – frame – delimiter; the delimiter is called the **flag**.
- **Byte stuffing**: put an ES before every FG and every ES in the payload; the receiver removes the first ES of the pair.
- **Bit stuffing**: the flag is 01111110; insert a 0 after five consecutive 1s, and remove it at the receiver.
- Throughout this chapter it is assumed there is **no noise** — framing solves ambiguity, not corruption.

CHAPTER 06

ERROR CONTROL AND FLOW CONTROL

Framing tells the receiver where a frame ends. Two things can still go wrong: the frame may arrive damaged, and it may arrive faster than the receiver can absorb. These are the two remaining data link services — error control and flow control. This chapter develops them from the simplest possible protocol up to Stop-and-Wait ARQ.

6.1 Error control: why errors happen

CAUSE	WHAT HAPPENS
(i) Channel noise	Electrical interference flips one or more bits while the frame is in transit, so the frame arrives corrupted.
(ii) Two messages sent together	Two stations transmit at the same instant on a shared medium; the signals overlap and both frames are destroyed.

WHERE THIS LEADS

The second cause is the reason the CSMA/CD family of protocols exists — see Chapters 9 and 11. Error control, discussed here, deals with the **consequences**; medium access control tries to **prevent** the collision in the first place.

CRC — CYCLIC REDUNDANCY CHECK

The sender treats the frame as a long binary number, divides it by an agreed generator polynomial, and appends the remainder to the frame. The receiver repeats the division over the whole received frame: a zero remainder means the frame is almost certainly intact, a non-zero remainder means it is corrupted. CRC only **detects** the error; recovering from it is the job of the ARQ protocols below.

6.1.3 Lost versus corrupted frames

QUESTION ASKED IN CLASS

What is the difference between a lost frame and a corrupted frame? A corrupted frame does arrive at the receiver, but the CRC fails, so the receiver knows something came and was damaged, and discards it. A lost frame never arrives at all — the receiver has no idea that anything was sent.

From the sender's point of view, however, the two are identical: in both cases **no acknowledgement comes back**. This is why a single mechanism — the timeout — can recover from both.

6.2 Flow control

The sender holds its outgoing frames in a queue; the receiver holds arriving frames in a buffer until the higher layer collects them. Flow control exists because those two rates need not match.

Sender $\Rightarrow$ queue $\longrightarrow$ channel $\longrightarrow$ Receiver $\Rightarrow$ buffer

6.2.1 The Simplest Protocol. The first protocol is literally named the *Simplest Protocol*. It has no flow control and no error control at all: the sender simply sends frames one after another as fast as it likes, and the receiver is assumed to consume them. No noise is assumed.

PROBLEM

If the buffer on the receiver side becomes full, the receiver has no choice but to **drop** the frames that keep arriving. Data is lost even though the channel itself was perfectly good.

This is exactly the problem that flow control solves: the receiver must be given a way to tell the sender to wait.

6.2.2 The Stop-and-Wait Protocol. The fix is to make the sender wait for permission. In Stop-and-Wait the sender transmits one frame and then stops until the receiver returns an **ACK** (acknowledgement). The ACK is the receiver's way of saying "I have room, send the next one".

6.3 Stop-and-Wait ARQ

ARQ – AUTOMATIC REPEAT REQUEST

Stop-and-Wait handles flow control only, under the assumption that nothing goes wrong. Stop-and-Wait ARQ adds **error control** to it: every frame that is not acknowledged in time is automatically sent again. Two new ingredients are needed — a **timer** at the sender and a **sequence number** on every frame and every ACK.

THE TIMEOUT RULE

If the ACK is not received although the data was sent, then when the timer expires the data will be sent again. The sender cannot tell whether the data was lost, the data was corrupted or the ACK was lost — and it does not need to, because retransmission is the correct answer in every case.

WHY SEQUENCE NUMBERS ARE ESSENTIAL

Without a number on the frame, the receiver could not tell a retransmission from a genuine new frame, and a lost ACK would silently duplicate the data. In Stop-and-Wait ARQ **one bit is enough**, so the frames are numbered 0, 1, 0, 1, ...

6.3.3 What the number in an ACK means

NOTATION

ACK x — **x is the frame number being asked of the sender**, that is, the number of the *next* frame the receiver wants.

So ACK 1 does not mean “frame 1 has arrived”. It means “everything up to frame 0 has arrived correctly; now send me frame 1”. Reading acknowledgements this way is essential before moving on to the sliding-window protocols in the next chapter.

CAREFUL — THIS CONVENTION IS NOT UNIVERSAL

This cumulative reading is the one used by Stop-and-Wait ARQ and by Go-Back-N. **Selective Repeat acknowledges each frame individually**, so there ACK 3 means “frame 3 itself has arrived” and says nothing about frames 0, 1 and 2. Section 7.3 returns to this point; confusing the two conventions is the commonest mistake in this topic.

PROTOCOL	FLOW CONTROL	ERROR CONTROL	FRAMES IN FLIGHT
Simplest Protocol	No	No	Unlimited
Stop-and-Wait	Yes	No	1
Stop-and-Wait ARQ	Yes	Yes	1

CHAPTER 06 AT A GLANCE

- Errors arise from **channel noise** and from **two messages being sent together**. CRC detects them.
- A corrupted frame arrives damaged; a lost frame never arrives. To the sender both look the same — no ACK.
- Simplest Protocol: no control at all; the receiver’s buffer fills up and frames are dropped → the need for flow control.
- Stop-and-Wait: one frame outstanding, released by an ACK — flow control solved.
- Stop-and-Wait ARQ: adds a **timeout** and **sequence numbers** — error control solved; handles DATA corrupt and ACK corrupt alike.
- ACK x means “send me frame x next”.

CHAPTER 07

SLIDING WINDOW PROTOCOLS: GO-BACK-N AND SELECTIVE REPEAT

Stop-and-Wait is safe but slow — the sender is idle for a whole round trip after every frame. The sliding-window protocols allow several frames to be outstanding at once. This chapter covers Go-Back-N and its refinement, Selective Repeat ARQ, together with the window-size rule and the negative acknowledgement.

7.1 Sequence numbering and windows

Because more than one frame is now in flight, each frame must carry a **sequence number**, and the sender and receiver must each keep a **window** — the set of frame numbers they are currently willing to handle.

- **Sender window (sender buffer)** — the frames the sender may transmit without waiting for an acknowledgement. In the example used here the sender window size is 4.
- **Receiver window** — the frames the receiver is prepared to accept. In Go-Back-N the receiver window is always 1; it will accept only the exact frame it is expecting.

$$W_s \leq 2^m - 1 \quad m = \text{number of bits used for the sequence number}$$

With $m = 3$ bits the frame numbers run from 0 to 7, so the sender window may be at most 7. The numbers then wrap round and are re-used.

QUESTION ASKED IN CLASS

Why cannot the numbering be just 1 and 0, like Stop-and-Wait? Because several frames are outstanding at the same time. With only the numbers 0 and 1, the third frame in a burst would again be called 0, and the receiver could not tell whether an arriving frame numbered 0 is the new third frame or a retransmission of the first. The numbering must be wide enough to keep every frame in the window distinct — hence m bits giving 2^m numbers, of which at most $2^m - 1$ may be in the window at once.

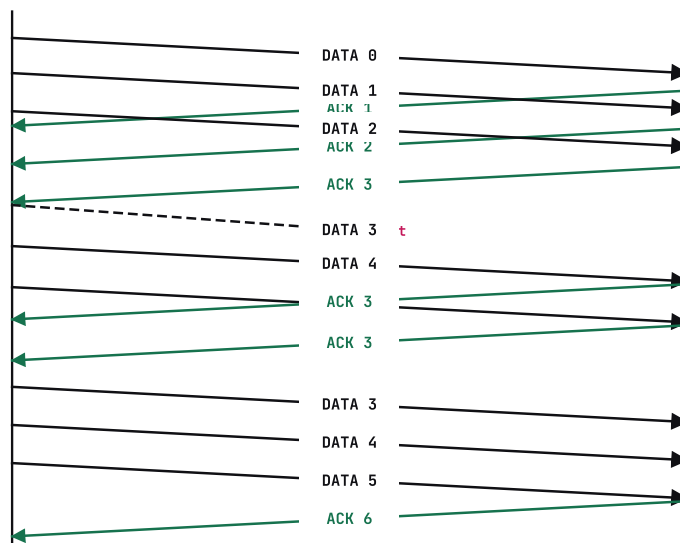
7.2 Go-Back-N

In Go-Back-N the sender transmits up to W_s frames without stopping. The receiver, having a window of 1, accepts frames **strictly in order** and discards anything out of order. When a frame goes missing, the sender must therefore go back and resend that frame **and every frame after it**.

DIAGRAM

GO-BACK-N — WINDOW 4
SENDER

RECEIVER



DATA 3 is lost. 4 and 5 do arrive, but the receiver window is 1, so it is still waiting for 3 and throws them away — each time repeating ACK 3. When the timer for 3 expires the sender goes back and resends 3, 4 and 5.

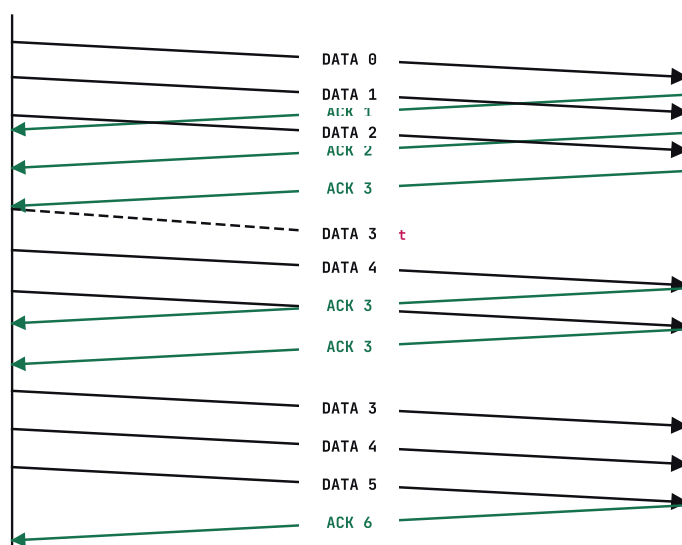
Fig. 7.1 Go-Back-N, case 1 — a frame in the middle of the window is lost. Frames 0, 1 and 2 arrive and are acknowledged (ACK 1, ACK 2, ACK 3 — each asking for the next frame). DATA 3 is lost, no ACK 4 comes back, and the timer expires. The rule is “go back to the lost frame and resend it and everything after it”; here frame 3 was the last of the window, so only frame 3 goes again.

7.2.2 Case 2 — the first frame of the window is lost. This is the case that gives the protocol its name.

DIAGRAM

GO-BACK-N — WINDOW 4
SENDER

RECEIVER



DATA 3 is lost. 4 and 5 do arrive, but the receiver window is 1, so it is still waiting for 3 and throws them away — each time repeating ACK 3. When the timer for 3 expires the sender goes back and resends 3, 4 and 5.

Fig. 7.2 DATA 0 is lost. Frames 1, 2 and 3 do arrive, but the receiver window is 1, so it is still expecting DATA 0 and discards them; it keeps replying ACK 0 — “still send me 0”. Those repeated acknowledgements never advance, so the sender learns the receiver did not even get DATA 0; formally the frame-0 timer expires, and the sender goes back and sends the whole window again, frames 1 to 3 included even though they had already arrived.

WHY THE RECEIVER WINDOW IS 1

Since the receiver will not store out-of-order frames, it has nowhere to put frames 1, 2 and 3 while frame 0 is missing. It therefore throws them away and repeats ACK 0. For the receiver, Go-Back-N behaves exactly like Stop-and-Wait — one frame at a time, strictly in order. All the parallelism is on the sender's side.

THE COST

Frames 1, 2 and 3 were delivered perfectly and are still discarded and retransmitted. When the error rate is high, most of the retransmitted traffic is frames that had already arrived safely.

7.3 Selective Repeat ARQ

Selective Repeat ARQ is a better version of Go-Back-N. The improvement is a single change: **the receiver is given a window larger than 1**, so it can buffer out-of-order frames. Consequently only the lost frame is sent again, not the entire set of preceding frames.

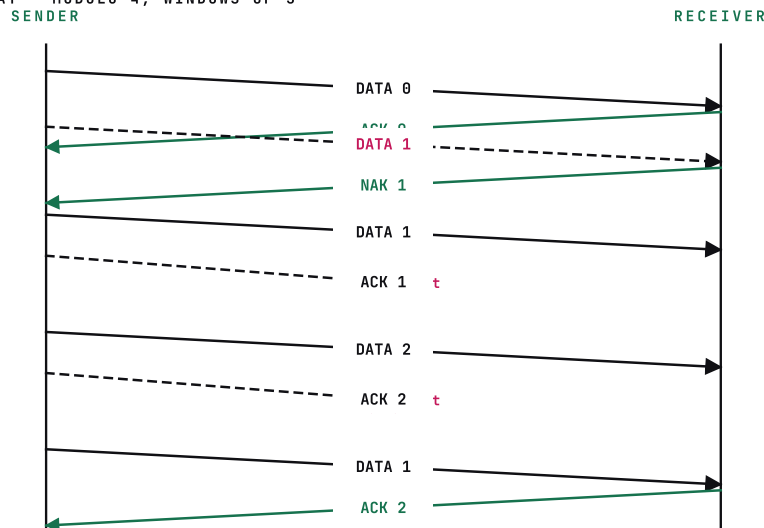
THE ACKNOWLEDGEMENT CHANGES MEANING HERE

Buffering out-of-order frames forces a second change that is easy to miss. In Go-Back-N the receiver could only ever acknowledge cumulatively, because it accepted frames strictly in order. A Selective Repeat receiver is holding frames 1, 2 and 3 while frame 0 is still missing, so a cumulative acknowledgement could not describe that state at all.

- **Go-Back-N:** ACK n = “everything up to $n - 1$ is safe, send me n next” — *cumulative*.
- **Selective Repeat:** ACK n = “frame n itself has arrived” — *individual*, and it says nothing about any other frame.
- For the same reason the sender runs a **separate timer for every outstanding frame**, not one timer for the whole window.

DIAGRAM

SELECTIVE REPEAT – MODULO 4, WINDOWS OF 3



A: frame 0 goes and is acknowledged. B: frame 1 arrives damaged, so the receiver sends NAK 1. C: frame 1 is resent and arrives, but its ACK is lost. D: frame 2 arrives, its ACK is lost too. E: the timer for 1 expires and only frame 1 is resent – that is the whole point of Selective Repeat. F: the next frame out is DATA 3.

Fig. 7.3 Selective Repeat. Frames 1, 2 and 3 are kept in the receiver's buffer and acknowledged individually. Only the missing frame 0 is retransmitted; the flow then continues with frames 4 and 5.

7.3.1 The negative acknowledgement

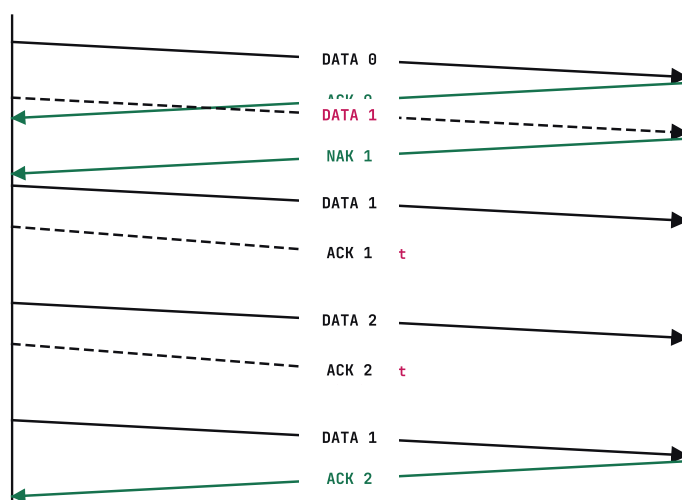
NACK – NEGATIVE ACKNOWLEDGEMENT

Instead of waiting for the sender's timer to expire, the receiver actively reports the gap: NACK 2 means “frame 2 is missing or damaged, send it again”. Recovery therefore starts at once rather than after a full timeout.

DIAGRAM

SELECTIVE REPEAT — MODULO 4, WINDOWS OF 3
SENDER

RECEIVER



A: frame 0 goes and is acknowledged. B: frame 1 arrives damaged, so the receiver sends NAK 1. C: frame 1 is resent and arrives, but its ACK is lost. D: frame 2 arrives, its ACK is lost too. E: the timer for 1 expires and only frame 1 is resent — that is the whole point of Selective Repeat. F: the next frame out is DATA 3.

Fig. 7.4 DATA 2 arrives corrupted, so the receiver sends NACK 2 and frame 2 comes back at once. Frame 0 arrived safely but its acknowledgement was lost, so nothing but frame 0's own timer can rescue it — which is why Selective Repeat needs one timer per outstanding frame.

QUESTION ASKED IN CLASS

What about ACK 0 and ACK 1 in the diagram above? ACK 1 gets through, so frame 1 is settled and the sender stops its timer for frame 1. ACK 0 is lost, and nothing later repairs it. Here acknowledgements are individual, not cumulative, so the ACK 3 that arrives afterwards confirms frame 3 only — it says nothing whatever about frame 0. As far as the sender can tell, frame 0 is still outstanding.

Frame 0 is therefore recovered by the only mechanism left: its own timer expires and the sender retransmits frame 0. The receiver already holds frame 0, so it discards the duplicate and simply re-sends ACK 0. That is precisely why a Selective Repeat sender must keep **one timer per outstanding frame** instead of the single window timer of Go-Back-N.

COMMON EXAM TRAP

If the same question were asked about Go-Back-N, the answer would be the opposite: there ACK 3 would cover frames 0, 1 and 2, because Go-Back-N acknowledgements are cumulative. Always check which protocol is being asked about before deciding what a lost ACK costs.

7.3.2 NACK and ACK sent together. The receiver need not send its reports one at a time. When a gap is noticed while later frames are still arriving, the negative acknowledgement for the missing frame and the positive acknowledgement for the frames that did arrive can be sent together.

AND IF NOTHING COMES BACK AT ALL

The timeout remains the last line of defence. If the ACK is not received although the data was sent, the data will be sent again when the timer expires — this covers the case where even the NACK is lost.

7.4 Comparison

PROTOCOL	WS	WR	MEANING OF ACK N	TIMERS	ON AN ERROR, RETRANSMIT	NACK
Stop-and-Wait ARQ	1	1	Cumulative — send me n next	1	The single outstanding frame	No
Go-Back-N	$2^m - 1$	1	Cumulative — all up to n - 1 are safe	1 for the window	The lost frame and all frames after it	No
Selective Repeat ARQ	$2^{(m-1)}$	> 1	Individual — frame n itself arrived	One per frame	Only the lost frame	Yes

EXAM TIP

Trace the logic rather than memorising the pictures: receiver window = 1 forces in-order acceptance, which forces the whole window to be resent (Go-Back-N). Enlarge the receiver window so it can buffer, and only the missing frame needs to come back (Selective Repeat). Everything else follows from that one difference.

CHAPTER 07 AT A GLANCE

- Sender window limit: $W_s \leq 2^m - 1$, m = number of sequence-number bits; with 3 bits the frames are numbered 0–7.
- Numbering cannot be just 0 and 1 once more than one frame is in flight.
- **Go-Back-N**: $W_s = 4$ in the example, $W_r = 1$; out-of-order frames are discarded and the receiver keeps repeating the ACK for the frame it wants; the whole window is resent.
- **Selective Repeat ARQ**: the receiver buffers out-of-order frames, so only the lost frame is resent.
- ACK n is **cumulative** in Go-Back-N but **individual** in Selective Repeat, which is why Selective Repeat needs one timer per outstanding frame.
- **NACK** lets the receiver report a gap immediately; NACK and ACK may be sent together (NACK 0 with ACK 1). The timeout still backs everything up.

MEDIUM ACCESS CONTROL: CONTROLLED ACCESS

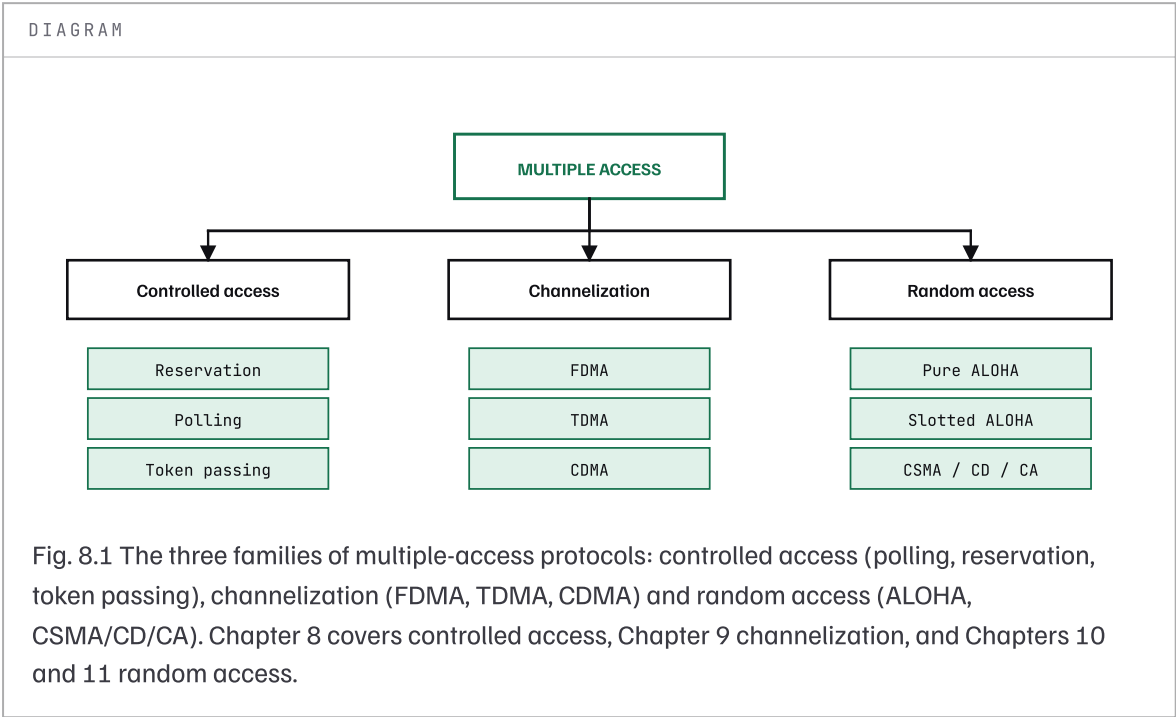
When many stations share one medium, something must decide whose turn it is — otherwise two messages sent together destroy each other. This chapter sets out the three families of medium access protocols and then covers the first family in full: polling, reservation and token passing, in which an authority explicitly grants permission to transmit.

8.1 Why medium access control is needed

Recall the two causes of error from Chapter 6. The first, channel noise, is a property of the medium. The second — two messages sent together — is entirely avoidable, and it is what medium access control exists to prevent.

CSMA / CD

Carrier Sense Multiple Access with Collision Detection. **Carrier sensing** means a station listens to the medium before it transmits; **multiple access** means many stations share the one medium; **collision detection** means a station notices when its transmission has been damaged by another. It is treated in detail in Chapter 11.



THE IDEA BEHIND CONTROLLED ACCESS

In controlled access, no station transmits on its own initiative. Permission is **granted** — either by a central station or by a circulating token — so a collision can never occur.

8.2 Polling

In polling one station is designated the **primary**; all others are **secondaries**. Every exchange goes through the primary. The primary polls each system at a time, asking “Do you have any data to send?”

8.2.2 Case (b) — the primary has data for a secondary. Here the primary must first tell the station to get ready. The operation that does this is called **SELECT**.

8.2.3 Case (c) — one secondary sending to another. Two secondaries never talk to each other directly. Communication from A to C is simply the two previous cases placed back to back.

$$(a) A \rightarrow \text{PRIMARY} + (b) \text{PRIMARY} \rightarrow C = A \rightarrow C$$

LIMITATION

Every frame crosses the medium twice and the primary is a single point of failure and a bottleneck. Polling is therefore only effective for **wired networks**, where the primary can reach every station reliably.

8.3 The reservation protocol

Polling asks each station in turn, which wastes a poll on every station that has nothing to send. The reservation protocol replaces the asking with a booking round, and it is effective for **wireless networks**.

- 1 Reservation mini-slots.** Time is divided into a short reservation interval containing one tiny slot per station. A station that wants to transmit marks its own mini-slot; a station with nothing to send leaves its mini-slot empty.
- 2 Scheduled DATA slots.** Every station has heard the whole reservation interval, so every station knows who has booked. Full-length data slots then follow, allocated in order to exactly those stations that reserved.
- 3 Station queues.** Each station keeps its own queue of pending frames and takes one frame from that queue per allotted data slot; whatever remains waits for the next reservation round.

8.4 Token passing

In token passing there is no primary at all. A special short frame — the **token** — circulates among the stations. Only the station currently holding the token may transmit; when it has finished, it passes the token on to its successor.

NOTE

Token passing is also called a **controlled access protocol**. It does not necessarily have to be a ring network — it can be a bus as well. What matters is not the physical shape but the agreed **logical order** in which the token is handed on.

8.5 Summary of the controlled-access methods

METHOD	WHO GRANTS PERMISSION	OVERHEAD	BEST SUITED TO
Polling	A central primary station	A poll is spent even on idle stations	Wired networks
Reservation	The stations themselves, in mini-slots	One short reservation interval per cycle	Wireless networks
Token passing	Whoever currently holds the token	The token must circulate even when idle	Ring or bus networks

EXAM TIP

All three methods share one guarantee: **no collision is possible**, because at any instant exactly one station is entitled to transmit. The price is the overhead of granting that entitlement — polls, mini-slots or a circulating token — which is paid even when the network is idle.

CHAPTER 08 AT A GLANCE

- Multiple-access protocols fall into three families: **controlled access**, **channelization** and **random access**.
- Polling**: primary and secondaries. Secondary→primary is POLL, DATA, ACK, SWITCH; primary→secondary is SELECT, DOWNLINK, ACK, POLL, NULL.
- $A \text{ to } C = (A \text{ to primary}) + (\text{primary to } C)$. Polling is only effective for wired networks.
- Reservation protocol** (effective for wireless): reservation mini-slots → scheduled data slots → station queues.
- Token passing** is also called a controlled access protocol; the network need not be a ring, it can be a bus.

CHAPTER 09

CHANNELIZATION: FDMA, TDMA AND CDMA

Controlled access lets stations take turns. Channelization takes the opposite approach: the single shared link is divided so that every station has a private share of it and no turn-taking is needed at all. The division can be made in frequency (FDMA), in time (TDMA) or in code (CDMA).

9.1 FDMA — Frequency Division Multiple Access

DEFINITION

The bandwidth of the link is cut into narrower frequency bands, and each station is permanently given one band. **Every node can put data into the channel at the same time** — a station does not have to wait for another node to finish sending, because the two are sending at different carrier frequencies.

$$C = B \times \log_2(1 + \text{SNR})$$

B = bandwidth of the band allotted to the station

SNR = signal-to-noise ratio

The formula makes the central weakness of FDMA obvious: the capacity a station gets is directly proportional to B, and B is only a fraction of the whole link.

LIMITATIONS OF FDMA

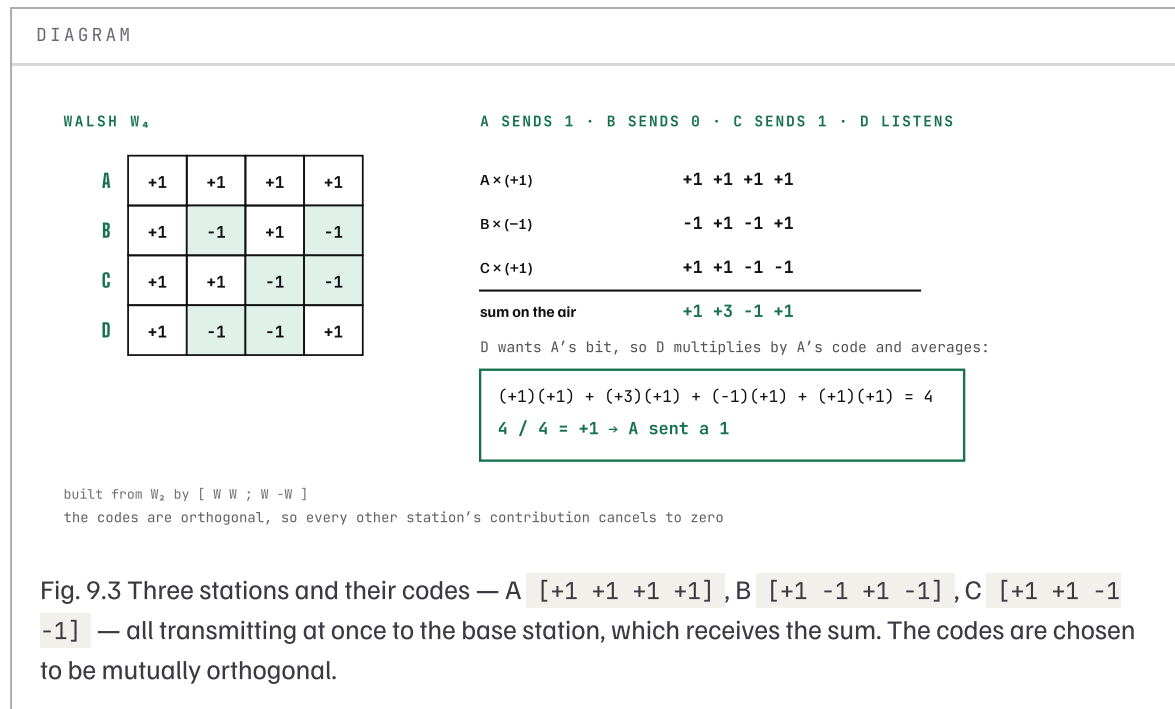
1. Capacity falls. Because we divided the link, each station now works with a small B and therefore a small capacity. If there are 4 nodes, the channel is divided by 4, so the data rate is also divided by 4. **2. No station can borrow another band.** A node cannot use node B's channel to increase its own data rate, even when node B is completely idle. The allocation is fixed, so an idle band is simply wasted.

9.2 TDMA — Time Division Multiple Access

TDMA divides the link in time instead of in frequency. Every station uses the **whole bandwidth**, but only during its own repeating time slot.

9.3 CDMA — Code Division Multiple Access

CDMA divides the link neither in frequency nor in time. Every station is given a distinct **code** — a short sequence of +1 and -1 values — and **all nodes can utilise the entire channel fully**, without doing TDM. All stations transmit at the same time over the same band, and the codes keep the signals separable.



ENCODING RULE

To send a **1**, the station transmits its code as it is, that is code × (+1). To send a **0**, the station transmits the **inverted** code — zero is sent as (-1), so it transmits code × (-1).

STATION	CODE	BIT SENT	WHAT GOES ON THE CHANNEL
A	[+1 +1 +1 +1]	1	[+1 +1 +1 +1]
B	[+1 -1 +1 -1]	0	[-1 +1 -1 +1]
C	[+1 +1 -1 -1]	1	[+1 +1 -1 -1]

9.3.2 What the base station receives. All three stations transmit at the same time, so the channel simply carries the **sum** of the three sequences, chip by chip.

CONTRIBUTION	CHIP 1	CHIP 2	CHIP 3	CHIP 4
User A (sent 1)	+1	+1	+1	+1
User B (sent 0)	-1	+1	-1	+1
User C (sent 1)	+1	+1	-1	-1
Received	1	3	-1	1

DECODING RULE

To recover what one station sent, multiply the received sequence with the code of that station **chip by chip**, add up the products, and divide by the number of chips. Doing this separately for A, B and C decodes all three from the same received sequence.

$$d = (\text{received} \cdot \text{code}) / N$$

a positive result means the bit was 1

a negative result means the bit was 0

DECODE FOR	RECEIVED · CODE, CHIP BY CHIP	SUM	÷ 4	BIT
A [+1 +1 +1 +1]	+1, +3, -1, +1	+4	+1	1
B [+1 -1 +1 -1]	+1, -3, -1, -1	-4	-1	0
C [+1 +1 -1 -1]	+1, +3, +1, -1	+4	+1	1

WHY IT WORKS – THE CHECK ON A SINGLE STATION

A code multiplied by itself gives +1 in every position, so the sum is the full length of the code:

$$[+1 \ -1 \ +1 \ -1] \times [+1 \ -1 \ +1 \ -1] = [+1 \ +1 \ +1 \ +1] = 4$$

while a code multiplied by **any other** station's code sums to zero. That is what makes the other stations disappear from the calculation, leaving only the wanted one.

A PRACTICAL POINT FROM THE LECTURE

Even if noise is introduced, it does not affect the result that much — the summation over the whole code averages the noise away, whereas the wanted signal adds up coherently.

The one problem is at the other end: if nothing is actually sent and there is noise, we still receive something — a small non-zero number. So in a real-world implementation a **threshold** such as 0.5 or 0.25 is set, and any decoded value below the threshold is treated as “nothing sent”. This is how the receiver knows what is merely its own noise.

9.4 Comparison

SCHEME	DIVISION IS BY	BANDWIDTH PER STATION	TIME AVAILABLE	IDLE STATION'S SHARE
FDMA	Frequency (carrier)	B/N	All the time	Wasted
TDMA	Time (slots)	Full B	1 slot in N	Wasted
CDMA	Code (orthogonal sequences)	Full B	All the time	Not reserved at all

9.5 Timing parameters

Three time quantities are used repeatedly in the access protocols of the next two chapters.

SYMBOL	NAME	MEANING
T_p	Maximum propagation time	The longest time a signal takes to travel between any two stations on the medium.
T_{fr}	Average transmission time for a frame	The time a station needs to put one complete frame onto the medium.
T_B	Backoff time	The random time a station waits after a collision, before trying again.

CHAPTER 09 AT A GLANCE

- **FDMA:** different carrier frequencies; everybody sends at once. Capacity = $B \log_2(1 + \text{SNR})$; 4 nodes $\rightarrow$ data rate divided by 4; an idle band cannot be borrowed.
- **TDMA:** full bandwidth, but only in the station's own time slot.
- **CDMA:** orthogonal codes; the whole channel is used fully by everybody at once. Send 1 as the code, 0 as the inverted code (-1).
- Decode by multiplying the received sequence with each station's code and dividing by the code length: positive $\rightarrow$ 1, negative $\rightarrow$ 0.
- Worked example: received [1, 3, -1, 1] decodes to $A = 1, B = 0, C = 1$.
- Parameters: T_p maximum propagation time, T_{fr} average frame transmission time, T_B backoff time.

CHAPTER 10

RANDOM ACCESS: PURE AND SLOTTED ALOHA

Controlled access and channelization both assume a fixed, known set of stations. Real networks are not like that. Random-access protocols let a station transmit whenever it likes and deal with the collisions afterwards. This chapter develops Pure ALOHA, its backoff procedure and its throughput, and then shows how a single change — slotting time — doubles that throughput.

10.1 Why a random-access protocol is needed

LIMITATION OF THE SCHEMES SEEN SO FAR

As the number of devices increases, polling, reservation and channelization all become very inefficient — every station must be given a poll, a mini-slot, a frequency band or a time slot, whether it has anything to send or not. The difficulty is worse still **at runtime**, when nodes get added or removed: the allocation has to be recomputed and redistributed every time the membership changes.

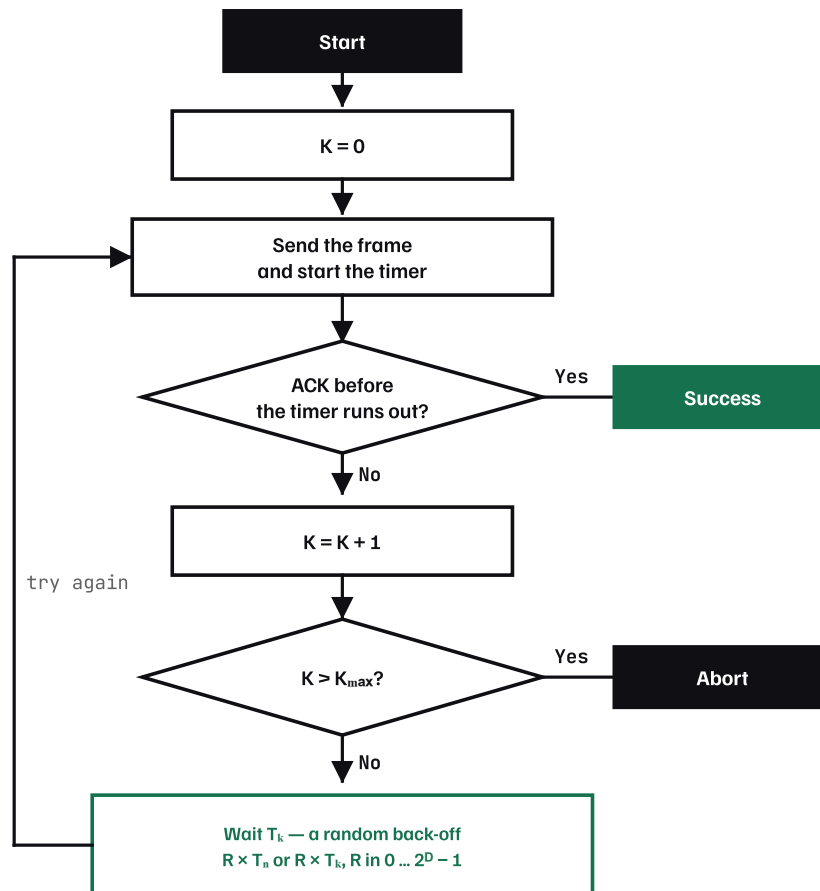
THE RANDOM-ACCESS IDEA

Give no station a fixed allocation. A station with a frame simply sends it. If it collides, the station waits a random time and tries again. Nothing in the protocol depends on how many stations there are, so stations can join and leave freely.

10.2 Pure ALOHA

Pure ALOHA is the simplest random-access protocol: transmit whenever you have data, wait for an acknowledgement, and if none comes, back off and retry.

DIAGRAM



K counts the attempts on the current frame. The wait is random on purpose — if every station waited the same time they would all collide again together.

Fig. 10.1 The Pure ALOHA procedure as a flowchart: send $\rightarrow$ wait $2 \times T_p \rightarrow$ ACK? $\rightarrow$ success, or $K = K + 1$, abort if $K > K_{\max}$, else pick a random R and wait T_B . K counts the attempts made for the current frame; $K_{\max}$ is the attempt limit, usually 15.

- 1 **Send the frame** as soon as it is ready — no sensing, no permission.
- 2 **Wait the time-out time, $2 \times T_p$** — one maximum propagation time for the frame to reach the far end and one for the acknowledgement to come back.
- 3 If the **ACK is received**, the transmission has succeeded.
- 4 If not, a collision is assumed: increment the attempt counter, **$K = K + 1$** .
- 5 If **$K > K_{\max}$** the station gives up and aborts. Otherwise it backs off.
- 6 Choose a random number **R between 0 and $2^K - 1$** , and wait **$T_B = R \times T_p$** (or $R \times T_{fr}$) before sending again.

BINARY EXPONENTIAL BACKOFF

R is drawn from the range 0 to $2^K - 1$, which **doubles at every failure**. The backoff window therefore keeps increasing as the collisions keep increasing. Two stations that collided are very unlikely to pick the same R twice, and a heavily loaded network automatically spreads its retries over a wider interval.

10.3 Vulnerable time

DEFINITION

The **vulnerable period** is the time interval during which, if another node sends a frame, a collision will occur with the frame this node is sending.

Vulnerable time (pure ALOHA) = $2 \times T_{fr}$

T_{fr} = the frame transmission time, i.e. the maximum time a frame occupies the channel

10.4 Throughput of Pure ALOHA

- **S — throughput.** S is the packet receiving ratio: the fraction of the frames that are sent which are actually received successfully.
- **G — offered load.** How much data went out, divided by how much data could have been sent.

$S = G e^{-2G}$ the exponent is $-2G$ because the vulnerable time is $2 T_{fr}$

BEST CASE

Differentiating $S = G e^{-2G}$ with respect to G and setting the result to zero gives a maximum at **G = 0.5**.

$$S_{\max} = 0.5 \times e^{-1} = 0.184$$

Meaning: out of 1000 frames that are sent, only about **184** can be received successfully. The remaining frames are lost in collisions and must be retransmitted.

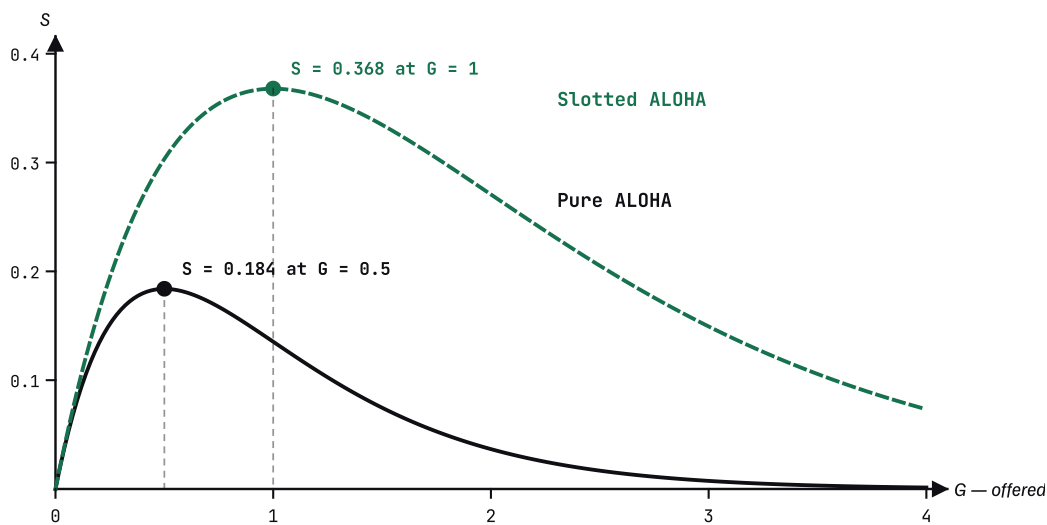
10.5 Slotted ALOHA

Pure ALOHA is vulnerable for two frame times because a frame can start at any instant. Slotted ALOHA removes half of that exposure with one rule: time is divided into slots of one frame time, and **a station may begin transmitting only at the start of a slot**.

$$S = G e^{-G}$$

$$S_{\max} = e^{-1} = 0.368 \quad \text{at } G = 1$$

DIAGRAM



past the peak, offering more traffic delivers less – the extra load is all collision

Fig. 10.4 Throughput S against offered load G , both curves on one set of axes: pure ALOHA peaking at 0.184 for $G = 0.5$ and slotted ALOHA at 0.368 for $G = 1$. Both rise, peak and then fall away — beyond the peak, extra offered load only produces extra collisions, so the useful throughput drops.

	PURE ALOHA	SLOTTED ALOHA
When may a station start?	At any instant	Only at a slot boundary
Vulnerable time	$2 T_{fr}$	T_{fr}
Throughput	$S = G e^{-2G}$	$S = G e^{-G}$
Best case at	$G = 0.5$	$G = 1$
Maximum throughput	0.184	0.368

10.6 Reservation compared with ALOHA

It is worth comparing the two approaches directly on the same network of 15 nodes.

	RESERVATION	ALOHA
Order of transmission	Sequential — the nodes are served in a fixed order	Random
Waiting for a turn	A node waits up to 15 turns to transmit fully. If one unit of time is one second, the first node's frame goes in the 1st second and the cycle takes 15 seconds	A node may need a minimum of about 15 attempts before it gets through, but it may also succeed immediately
When a node joins or leaves	The allocation must be changed accordingly	Nothing has to be changed

PROS OF ALOHA

Best when the number of nodes is **dynamic** — nodes may join or leave at any time and the allocation never has to be changed. Very practical, and simple to implement.

CONS OF ALOHA

Can be **slower** than reservation or the other controlled-access protocols, because time is lost in collisions and backoff. Throughput is low — at best 18.4% (pure) or 36.8% (slotted).

EXAM TIP

Remember the pair of formulas together with the reason for the exponent. Pure ALOHA is exposed for **two** frame times, so $S = G e^{(-2G)}$; slotted ALOHA is exposed for **one**, so $S = G e^{(-G)}$. Halving the vulnerable time exactly doubles the peak throughput, from 0.184 to 0.368.

CHAPTER 10 AT A GLANCE

- Random access is used because controlled schemes become inefficient as the number of devices grows and when nodes are added or removed at runtime.
- **Pure ALOHA:** send $\rightarrow$ wait $2 T_p \rightarrow$ ACK? $\rightarrow$ if not, $K = K + 1$, abort if $K > K_{\max}$ (usually 15), else wait $T_B = R \times T_p$ with R random in $[0, 2^K - 1]$.
- The backoff window keeps increasing as collisions keep increasing (binary exponential backoff).
- Vulnerable time $2 T_{fr}$; $S = G e^{(-2G)}$; $S_{\max} = \mathbf{0.184}$ at $G = 0.5$ — about 184 of every 1000 frames get through.
- **Slotted ALOHA:** vulnerable time T_{fr} ; $S = G e^{(-G)}$; $S_{\max} = \mathbf{0.368}$ at $G = 1$.

CHAPTER 11

CSMA AND ITS VARIANTS

ALOHA transmits blindly. The obvious improvement is to listen first. That single idea gives Carrier Sense Multiple Access, and from it the collision-detection and collision-avoidance variants used in wired and wireless LANs. This chapter also covers the two problems that carrier sensing cannot solve on its own — the hidden terminal and the exposed terminal.

11.1 CSMA — Carrier Sense Multiple Access

THE IDEA

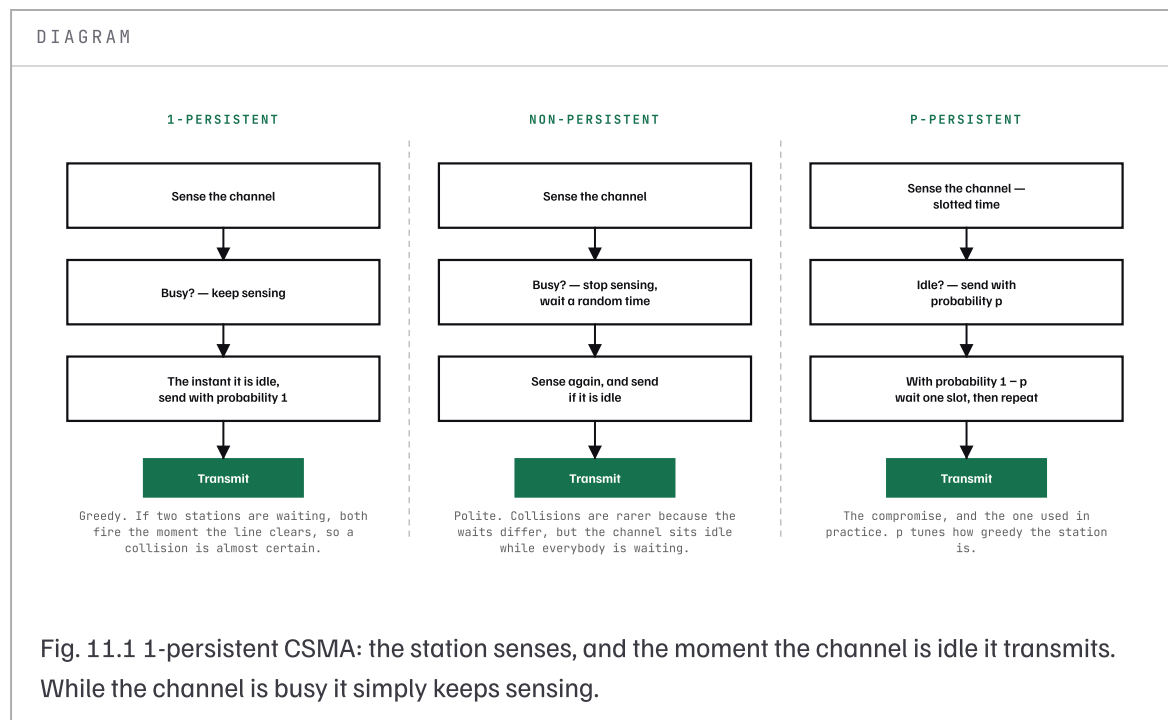
Before transmitting, a station **senses** the channel. If somebody else is already transmitting, it does not start. This removes every collision except those caused by two stations sensing an idle channel at almost the same moment. In short: channel sensing resolves most of the waste that ALOHA suffers.

MAKING THE CARRIER DETECTABLE

For sensing to work, the neighbouring stations must be able to recognise that a transmission has started before the real frame content begins. A fixed pattern of **56 bits** is therefore sent even before the flag, so the nodes around the transmitter can lock on to it. It also gives every receiver a reference time from which to measure the rest of the frame.

11.1.1 Persistent CSMA. Sensing tells a station that the channel is busy. It does not say what to do next, and that is where the persistence methods differ.

- **1-persistent** — the station keeps sensing continuously and transmits the instant the channel goes idle. Simple, but if two stations are both waiting they will both jump in at the same moment and collide.
- **p-persistent** — when the channel is found idle the station transmits only with probability p : it draws a random number and, if it is below the threshold p , it transmits; otherwise it waits for the next slot and tries the test again. Spreading the decision over several slots makes a simultaneous start much less likely.



11.2 CSMA/CD — Collision Detection

Sensing does not eliminate collisions altogether, because two stations can start within one propagation time of each other. CSMA/CD adds a second rule: **a station keeps listening while it transmits**. If it detects that the signal on the medium is not its own, a collision has occurred; it aborts the transmission immediately and backs off. The channel is thus freed at once instead of being wasted for the whole frame time. This is the method used on wired LANs.

11.3 CSMA/CA — Collision Avoidance

On a wireless medium a station **cannot listen while it transmits** — its own signal is far too strong — so collisions cannot be detected. They must instead be avoided in advance, which is what CSMA/CA does.

- 1 **Continuous sensing.** The station senses the channel until it finds it free.
- 2 **IFS — Inter-Frame Space.** Even after the channel is found free, the station waits a fixed gap before doing anything. This lets a station that is entitled to priority (for example one sending an acknowledgement) go first, and it absorbs the case where a distant station has already started but its signal has not yet arrived.
- 3 **Contention window.** The station picks a random one of the window's slots and counts down to it, sensing all the while. The size of the window is **binary exponential** — it doubles after every failed attempt, exactly as in the ALOHA backoff.
- 4 **Send the frame.** If the channel is still free when the countdown finishes, the DATA frame is transmitted.
- 5 **Time-out.** The station waits for the acknowledgement. If none arrives within the time-out, the frame is treated as lost and the procedure restarts with a doubled window.

VALUES USED

IFS = inter-frame space time

slot = 60 us

IMPORTANT RULE – THE BACKOFF WINDOW IS NOT RESET

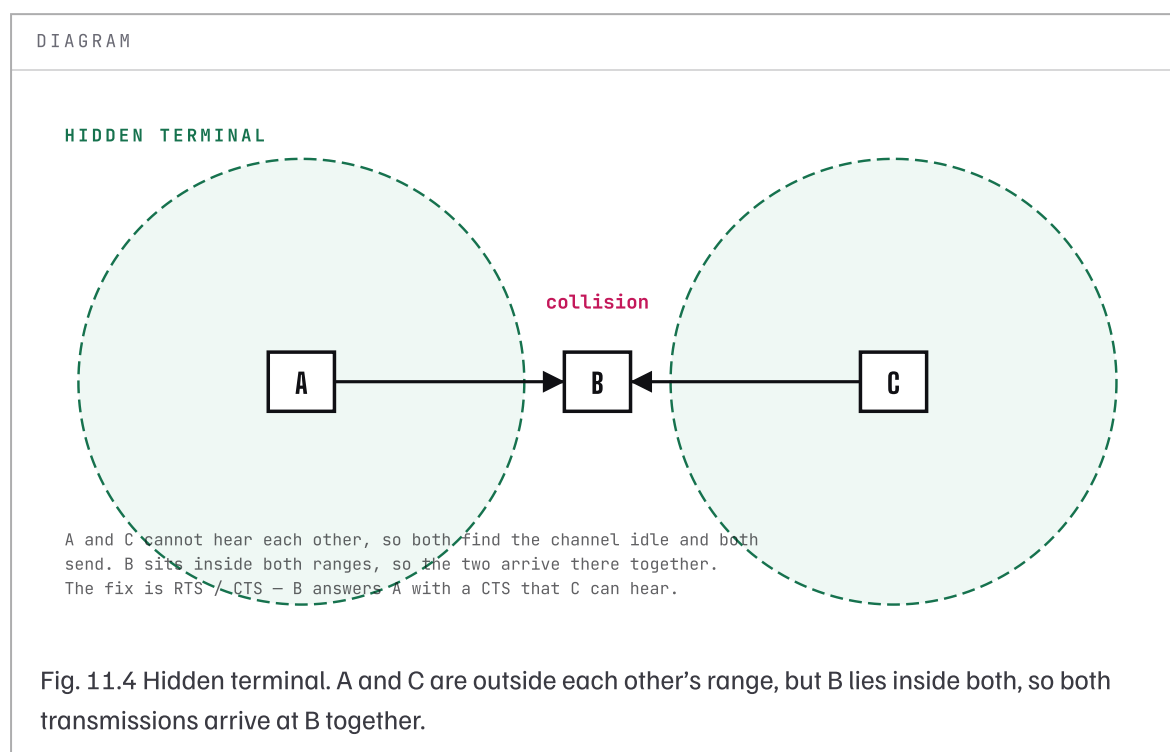
When one node is counting down its backoff window and, in between, a different node occupies the channel, the backoff window is **not reset**. The countdown is simply paused and continued from where it stopped once the channel is free again.

If the counter were restarted, a station that had already waited a long time would be sent to the back of the queue each time somebody else transmitted, and could be starved indefinitely. Freezing the counter guarantees that the longest waiter transmits first.

11.4 Two problems carrier sensing cannot solve

Carrier sensing assumes that a station can hear every other station. On a wireless medium that assumption fails, because each station has a limited transmission range.

11.4.1 Hidden terminal. A is talking to B, and at the same time C is talking to D.



THE PROBLEM – HIDDEN TERMINAL

A is talking to B and C is talking to D. C cannot hear A at all, so when C senses the channel it finds it **free** and transmits — and its signal collides with A's at B. Put from B's side: B is trying to detect the power level of A, but it detects that of C instead. The two transmissions arrive together and B receives neither correctly. A and C are hidden from each other, and carrier sensing gives them no warning.

DIAGRAM

EXPOSED TERMINAL

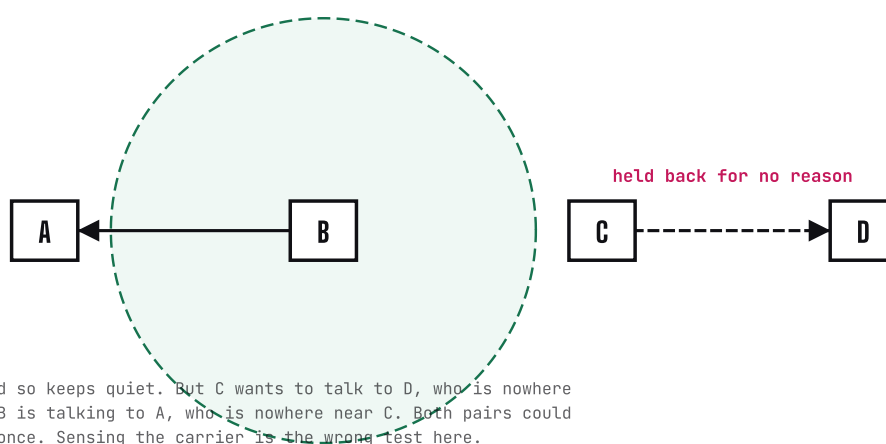


Fig. 11.5 Exposed terminal. C is inside B's range and so hears B, although C's own destination D is far away from B.

THE PROBLEM – EXPOSED TERMINAL

B wants to send data to A and begins transmitting. C senses that the link is occupied and therefore does **not** send its data to D. But this caution is unnecessary: if C could have sent, nothing would have gone wrong. C's transmission is aimed at D, which is outside B's range, and B's transmission is aimed at A, which is outside C's range. The two exchanges would not have interfered at all. C is needlessly exposed to B's signal, and capacity is wasted.

	HIDDEN TERMINAL	EXPOSED TERMINAL
What the sensing station concludes	Channel is free (wrongly)	Channel is busy (wrongly)
Consequence	It transmits and causes a collision	It stays silent and wastes an opportunity
Effect on the network	Lost frames	Lost capacity

EXAM TIP

Both problems come from the same cause — **carrier sensing happens at the sender, while collisions happen at the receiver**. A station senses the wrong place. That is why wireless protocols add an explicit handshake before the data, instead of relying on sensing alone.

CHAPTER 11 AT A GLANCE

- **CSMA**: sense before transmitting; 56 bits are sent before the flag so neighbouring nodes can detect the carrier and get a reference time.
- **Persistence**: 1-persistent keeps sensing and sends as soon as the channel is idle; p-persistent sends with probability p , otherwise waits for the next slot.
- **CSMA/CD**: listen while transmitting, abort on a collision — used on wired LANs.
- **CSMA/CA**: sense → IFS → contention window (binary exponential, slot $60\ \mu\text{s}$) → send frame → time-out.
- The backoff window is **not reset** when another node seizes the channel — it is paused and resumed.
- **Hidden terminal**: the sender wrongly believes the channel is free → collision at the receiver. **Exposed terminal**: the sender wrongly believes it is busy → wasted capacity.

THE WHOLE SUBJECT IN ONE SCREEN

EVERYTHING, CONDENSED

01 ADDRESSING IN A NETWORK

- Three addresses: **IP** host to host across networks, **MAC** node to node inside one LAN, **port** process to process.
- A MAC address cannot move data to a different network — that is the IP address's job, and a new MAC is used on every hop.
- A MAC address is **48 bits**, six hex octets, burned into the NIC.
- Classification: all ones → broadcast; otherwise bit 0 of the first octet, 0 → unicast, 1 → multicast.
- Bit 1 of the first octet: 0 → universal, 1 → locally administered.

02 SWITCHES, BRIDGES AND THE LOOP PROBLEM

- A switch holds a table of **MAC address** → **interface** and forwards packets according to it. Bridge and switch mean the same thing here.
- A **learning bridge** fills that table automatically from the source address of arriving frames; unknown destinations are flooded.
- Table entries are marked fresh and keep refreshing after some time, so that stale entries disappear.
- Two ports joining the same bridges form a **loop**, which causes infinite forwarding.
- **BPDU** (Bridge Protocol Data Unit) checks whether a loop exists and carries configuration and priority; only one port stays active and the other is blocked.

03 DATA TRANSMISSION AND NETWORK PARAMETERS

- The **bottleneck link** is the slowest link on the path; it fixes the usable rate of the whole path.
 - Offered load above the link capacity leads to packet drops; a small short-term excess is absorbed by the buffer.
 - Delay at a node = **processing + queueing + transmission + propagation**, and only queueing grows with load.
 - Queueing is necessary because two packets can arrive at the same time; without a queue the router cannot tell which is which and the data becomes invalid.
-

04 TRANSMISSION MEDIA

- Three primary ways of transmission: electrical signal, EMW, optical signal; inside the circuit it is always electrical.
 - Twisted pair / RJ45: +ve on one wire, -ve on the other; the receiver reads the **difference**, so common noise cancels.
 - Coaxial cable is a transmission line; Z_i , Z_o and the load must be matched (50Ω), otherwise reflections and loss occur.
 - Optical fibre: core (n_1) surrounded by cladding (n_2), with $n_2 < n_1$ giving total internal reflection.
 - Three fibre types: single mode, multimode step index, multimode graded index. Varying the launch angle carries multiple messages — possible only in multimode.
-

05 THE DATA LINK LAYER AND FRAMING

- Data link layer services: **framing**, **error control** (CRC), **flow control**.
 - A frame is delimiter – frame – delimiter; the delimiter is called the **flag**.
 - **Byte stuffing**: put an ES before every FG and every ES in the payload; the receiver removes the first ES of the pair.
 - **Bit stuffing**: the flag is 01111110; insert a 0 after five consecutive 1s, and remove it at the receiver.
 - Throughout this chapter it is assumed there is **no noise** — framing solves ambiguity, not corruption.
-

06 ERROR CONTROL AND FLOW CONTROL

- Errors arise from **channel noise** and from **two messages being sent together**. CRC detects them.
 - A corrupted frame arrives damaged; a lost frame never arrives. To the sender both look the same — no ACK.
 - Simplest Protocol: no control at all; the receiver's buffer fills up and frames are dropped → the need for flow control.
 - Stop-and-Wait: one frame outstanding, released by an ACK — flow control solved.
 - Stop-and-Wait ARQ: adds a **timeout** and **sequence numbers** — error control solved; handles DATA corrupt and ACK corrupt alike.
 - ACK x means "send me frame x next".
-

07 SLIDING WINDOW PROTOCOLS: GO-BACK-N AND SELECTIVE REPEAT

- Sender window limit: $W_s \leq 2^m - 1$, m = number of sequence-number bits; with 3 bits the frames are numbered 0–7.
- Numbering cannot be just 0 and 1 once more than one frame is in flight.
- **Go-Back-N**: $W_s = 4$ in the example, $W_r = 1$; out-of-order frames are discarded and the receiver keeps repeating the ACK for the frame it wants; the whole window is resent.
- **Selective Repeat ARQ**: the receiver buffers out-of-order frames, so only the lost frame is resent.
- ACK n is **cumulative** in Go-Back-N but **individual** in Selective Repeat, which is why Selective Repeat needs one timer per outstanding frame.
- **NACK** lets the receiver report a gap immediately; NACK and ACK may be sent together (NACK 0 with ACK 1). The timeout still backs everything up.

08 MEDIUM ACCESS CONTROL: CONTROLLED ACCESS

- Multiple-access protocols fall into three families: **controlled access**, **channelization** and **random access**.
- **Polling**: primary and secondaries. Secondary→primary is POLL, DATA, ACK, SWITCH; primary→secondary is SELECT, DOWNLINK, ACK, POLL, NULL.
- $A \text{ to } C = (A \text{ to primary}) + (\text{primary to } C)$. Polling is only effective for wired networks.
- **Reservation protocol** (effective for wireless): reservation mini-slots → scheduled data slots → station queues.
- **Token passing** is also called a controlled access protocol; the network need not be a ring, it can be a bus.

09 CHANNELIZATION: FDMA, TDMA AND CDMA

- **FDMA**: different carrier frequencies; everybody sends at once. Capacity = $B \log_2(1 + \text{SNR})$; 4 nodes → data rate divided by 4; an idle band cannot be borrowed.
- **TDMA**: full bandwidth, but only in the station's own time slot.
- **CDMA**: orthogonal codes; the whole channel is used fully by everybody at once. Send 1 as the code, 0 as the inverted code (−1).
- Decode by multiplying the received sequence with each station's code and dividing by the code length: positive → 1, negative → 0.
- Worked example: received [1, 3, -1, 1] decodes to $A = 1, B = 0, C = 1$.
- Parameters: T_p maximum propagation time, T_{fr} average frame transmission time, T_B backoff time.

10 RANDOM ACCESS: PURE AND SLOTTED ALOHA

- Random access is used because controlled schemes become inefficient as the number of devices grows and when nodes are added or removed at runtime.
 - **Pure ALOHA:** send $\rightarrow$ wait $2 T_p \rightarrow$ ACK? $\rightarrow$ if not, $K = K + 1$, abort if $K > K_{\max}$ (usually 15), else wait $T_B = R \times T_p$ with R random in $[0, 2^K - 1]$.
 - The backoff window keeps increasing as collisions keep increasing (binary exponential backoff).
 - Vulnerable time $2 T_{fr}$; $S = G e^{(-2G)}$; $S_{\max} = \mathbf{0.184}$ at $G = 0.5$ — about 184 of every 1000 frames get through.
 - **Slotted ALOHA:** vulnerable time T_{fr} ; $S = G e^{(-G)}$; $S_{\max} = \mathbf{0.368}$ at $G = 1$.
-

11 CSMA AND ITS VARIANTS

- **CSMA:** sense before transmitting; 56 bits are sent before the flag so neighbouring nodes can detect the carrier and get a reference time.
 - **Persistence:** 1-persistent keeps sensing and sends as soon as the channel is idle; p-persistent sends with probability p , otherwise waits for the next slot.
 - **CSMA/CD:** listen while transmitting, abort on a collision — used on wired LANs.
 - **CSMA/CA:** sense $\rightarrow$ IFS $\rightarrow$ contention window (binary exponential, slot $60 \mu s$) $\rightarrow$ send frame $\rightarrow$ time-out.
 - The backoff window is **not reset** when another node seizes the channel — it is paused and resumed.
 - **Hidden terminal:** the sender wrongly believes the channel is free $\rightarrow$ collision at the receiver. **Exposed terminal:** the sender wrongly believes it is busy $\rightarrow$ wasted capacity.
-

That is the entire revision set — 58 lines. If a line does not make sense, the chapter it came from is one tap away, and the **full course** is behind that.